SAGA pattern: a freight forwarding demo (NestJS microservices)

A customer asks a **forwarder** to move **2 containers from Shanghai to Rotterdam, then by truck to a warehouse in Germany**. To do that, the forwarder has to coordinate several independent companies. Each one is a separate service with **its own database**:

#	Service	Forward action (T_i)	Compensation (C_i)	Port
1	`ocean-carrier`	Book vessel slots CNSHA → NLRTM	Cancel ocean booking	4001
2	`customs`	File EU import declaration	Withdraw the declaration	4002
3	`trucking`	Book trucks Rotterdam → Germany	Cancel the truck order	4003
4	`warehouse`	Reserve pallet space	Release the space	4004
5	`billing`	Charge the customer (**pivot**)	Refund	4005
	`orchestrator`	The forwarder: HTTP API + saga engine		3000

Stack: NestJS 12 · `@nestjs/microservices` over **TCP** · TypeORM · SQLite (`data/<service>.sqlite`).

---

1. Why a saga?

In a monolith you would wrap all five bookings in one DB transaction: either everything commits or everything rolls back. With microservices that's impossible. There are 5 databases owned by 5 teams, and nobody offers a distributed transaction (2PC).

A **saga** replaces the single transaction with a **sequence of local transactions** T1 → T2 → … → Tn. Each service commits its own step immediately. If step Tk fails, the saga runs **compensating transactions** for the steps that already committed, **in reverse order**: Ck-1 → … → C1.

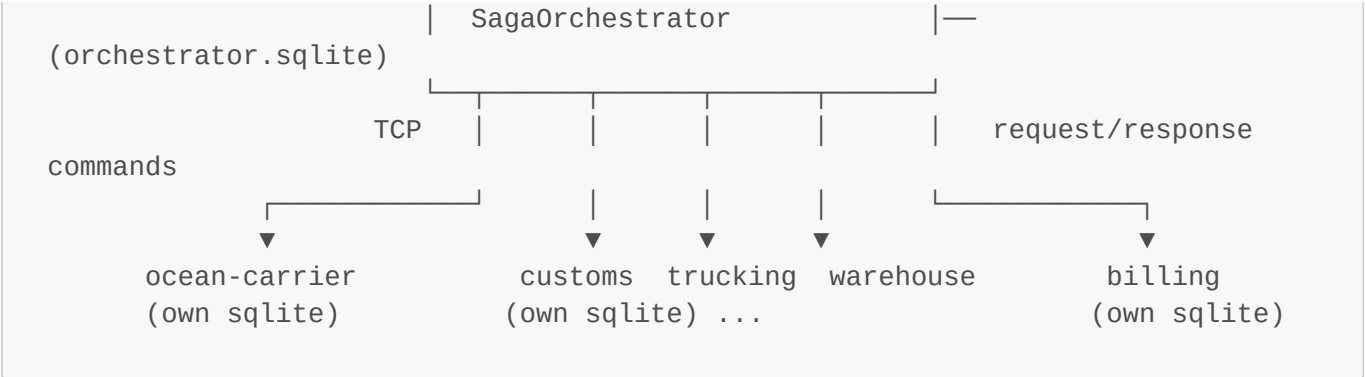
```
Happy path:   T1 ──▶ T2 ──▶ T3 ──▶ T4 ──▶ T5                          ✅
COMPLETED
Warehouse ✘:  T1 ──▶ T2 ──▶ T3 ──▶ T4✘
                                    └──▶ C3 ──▶ C2 ──▶ C1          ↩

COMPENSATED
```

A compensation is **not a rollback**. It's a new business action ("cancel the booking"), and the booking row stays in the database with status `CANCELLED`. The system is only **eventually consistent**: for a short while, the vessel slot is booked even though the saga is going to fail.

2. Orchestration (what this repo implements)

```
                                ┌──────────────────────────────┐
  POST /shipments ──▶ │  orchestrator (forwarder)    │   sagas +
saga_step_logs
```



A central orchestrator knows the steps and tells each participant what to do. Participants know nothing about each other or about the saga. They only expose "do X" and "undo X". (The alternative, **choreography**, has no central brain: services react to each other's events. That's a good next exercise.)

3. Concepts, and where to find them in the code

Concept	What it means	Where
Saga definition	The ordered list of (T_i, C_i) pairs	shipment-saga.definition.ts
Saga log / state	The orchestrator persists progress **after every step**, so it always knows what to undo	saga.entity.ts, saga-step-log.entity.ts
Execution + compensation engine	Runs T1..Tn; on failure runs the Cs in reverse	saga-orchestrator.service.ts
Saga context	Results of earlier steps feed later ones: trucking plans the pick-up from the vessel ETA, billing builds the invoice from every step's cost	`context` in `StepCommand`, see trucking.service.ts, billing.service.ts
Pivot transaction	Billing goes **last**: once money is charged nothing else can fail, so a refund is never needed in the normal flow. Put irreversible or expensive steps last.	shipment-saga.definition.ts
Idempotency	Each participant keys its row by `sagaId` (unique). A repeated "book" returns the existing booking; a repeated "cancel" returns `ALREADY_CANCELLED`. That makes retries safe.	every `*.service.ts` in the participants
Unknown outcome (timeouts)	A timeout does **not** mean failure; the work may have been done. The orchestrator then **also compensates the step that timed out**. A rejection or `ECONNREFUSED` means nothing happened, so that step is skipped.	`lastStepOutcomeUnknown` in the orchestrator, participant-clients.ts
Tombstones (out-of-order messages)	If a "cancel" arrives before the "book" (or for a booking that never happened), the participant stores a `CANCELLED` row. A late "book" for that saga is then rejected instead of creating an orphan booking.	`cancel()` in the participants

Concept	What it means	Where
Compensations must eventually succeed	Compensations are retried with backoff. If they still fail, the saga becomes `COMPENSATION_FAILED` and needs a human (`POST /shipments/:id/retry-compensation`).	`compensate()` / `retryCompensation()`
Crash recovery	On boot, sagas stuck in `STARTED`/`COMPENSATING` are recovered with **backward recovery** (undo everything that might have happened).	`recoverUnfinished()`, saga-recovery.service.ts

Saga states:

```
STARTED ──(all steps ok)──────────────────────────────► COMPLETED
   └─(a step fails)─► COMPENSATING ──(all undone)─────► COMPENSATED
                                └─(an undo keeps failing)─► COMPENSATION_FAILED
(manual retry)
```

4. Run it

Requires Node 20+ (tested on Node 24). No Docker, no broker.

```
npm install
npm run start:all        # all 6 apps in watch mode, colour-coded logs in one
terminal
```

Or use separate terminals so each service's logs are easier to follow:

```
npm run start:participants
npm run start:orchestrator
```

Then open **http://localhost:3000/** for the live UI (see below), or, in another terminal:

```
./scripts/demo.sh        # runs all the scenarios below
```

`requests.http` has the same calls for the VS Code REST Client. `npm run db:reset` wipes all databases (stop the apps first).

Live UI

The orchestrator serves a small page at **http://localhost:3000/** (apps/orchestrator/public/index.html, plain HTML/JS, no build step). Click a scenario and watch it:

- every command travel from the orchestrator to a participant and its reply come back (colour = outcome),
- the orchestrator's saga row (`status`, `completedSteps`, `lastStepOutcomeUnknown`) being saved after each step,
- the compensation plan run in reverse order, including retries,
- afterwards, that saga's row in **each participant's own DB** (e.g. `CANCELLED`, not deleted).

Use *Playback* to slow the animation down; events are queued, so nothing is lost. Past sagas can be replayed from the history list. It also shows which participants are reachable, so the "participant down" and "orchestrator crash" demos below are easy to follow.

The page is fed by `GET /events` (Server-Sent Events published by saga-events.ts). That bus is only for observability; the saga never depends on it.

5. Scenarios to try

Every request field is optional. `POST /shipments {}` sends 2×40HC of solar panels for `acme-gmbh` to Duisburg.

Scenario	Body	What you'll see
Happy path	`{}`	5× ✔, `COMPLETED`, invoice total in `context.billing`
Customs rejects	`{"hsCode":"930100"}`	fails at step 2 → cancel ocean
Not enough trucks	`{"containers":6}`	fails at step 3 → withdraw customs → cancel ocean
No warehouse in city	`{"finalDestinationCity":"Berlin"}`	fails at step 4 → 3 compensations in reverse
Credit limit	`{"customerId":"tiny-traders"}`	fails at the pivot → 4 compensations
Vessel full	`{"containers":12}`	fails at step 1 → nothing to undo
Force a failure	`{"simulate":{"failAt":"warehouse"}}`	any service can be told to reject
Timeout	`{"simulate":{"slowReplyAt":"trucking"}}`	trucking **commits** but replies after 8s; the orchestrator times out at 3s, and the compensations **include trucking**

Scenario	Body	What you'll see
Flaky compensation	`{"simulate": {"failAt":"billing","flakyCompensationAt":"customs"}}`	customs' withdraw fails twice, then succeeds on retry 3

Participant down → `COMPENSATION_FAILED` → manual retry

```
SAGA_STEP_DELAY_MS=2000 npm run start:orchestrator    # slow the saga down
curl -XPOST localhost:3000/shipments -H 'content-type: application/json' -d
'{"simulate":{"failAt":"trucking"}}'
# within ~3s, stop ocean-carrier (Ctrl+C its terminal). The saga ends
COMPENSATION_FAILED.
# start ocean-carrier again, then:
curl -XPOST localhost:3000/shipments/<id>/retry-compensation
```

Orchestrator crash → recovery on restart

```
SAGA_STEP_DELAY_MS=2000 npm run start:orchestrator
curl -XPOST localhost:3000/shipments -H 'content-type: application/json' -d
'{}'
# after ~5s, kill the orchestrator (Ctrl+C) while the saga is mid-flight,
then start it again.
# Logs: "found 1 unfinished saga(s) ... recovering". GET /shipments/<id>
shows it COMPENSATED.
```

6. API

Method	Path	
`POST`	`/shipments`	Start a saga; responds when it reaches a final state
`GET`	`/shipments`	List sagas
`GET`	`/shipments/:id`	Saga state + human-readable `timeline` + `context`
`GET`	`/shipments/:id/participants`	That saga's row in **each participant's own DB**, to confirm the compensations really happened
`POST`	`/shipments/:id/retry-compensation`	Retry a `COMPENSATION_FAILED` saga
`GET`	`/events`	Live saga events (Server-Sent Events), used by the UI
`GET`	`/saga-definition`	The ordered (T_i, C_i) steps
`GET`	`/services`	Which participants are reachable
`GET`	`/`	The live UI

Environment variables (orchestrator): `SAGA_STEP_TIMEOUT_MS` (3000), `SAGA_STEP_DELAY_MS` (0), `SAGA_COMPENSATION_MAX_ATTEMPTS` (5).

7. Project layout

```
apps/
  orchestrator/src/
    saga/           saga engine: definition, entities, TCP clients, recovery
    shipments/      HTTP controller, DTO validation, timeline view
  ocean-carrier/ customs/ trucking/ warehouse/ billing/
    src/            entity + service (business rules, do/undo) + TCP
controller + main.ts
libs/common/src/    shared contracts: ports, message patterns, command types,
failure simulation
data/               one SQLite file per service (created on first start)
```

8. Where to go next (exercises)

1. **Choreography**: remove the orchestrator and have each service emit events (`OceanBooked` → customs listens …). Compare how hard it is to follow the flow.
2. **Transactional outbox**: here, a participant's DB write and its reply aren't atomic. Add an `outbox` table and a relay.
3. **Semantic lock**: add a `PENDING` status so other sagas can't use a reservation until the saga completes.
4. **Forward recovery**: for retryable failures (e.g. trucking briefly down), retry the step instead of compensating.
5. **Async API**: return `202 Accepted` with the saga id and let clients poll `GET /shipments/:id`.